ocean Ai

// session 07 beginner Python

Give the ship a home

Bundle scattered data and behaviour into one object.

class self methods

// the object pivot

ocean Ai

Pass one ship, not four loose values.

Create one class, one object, and two methods. The game should play exactly as before.

01

Bundle

Name, crew, oxygen, and hull.

02

Behave

Status and defeat become methods.

03

Refactor

Use ship throughout the loop.

OUTCOME **You can define and use one meaningful class.**

// retrieve functions

Which part is the parameter?

```
def check_defeat(oxygen, hull):
```

oxygen and hull

They are local names for data passed into the function.

The functions are cleaner. The ship is still scattered.

How many times do the same values travel separately?

```
check_defeat(oxygen, hull)
```

```
show_status(oxygen, hull)
```

```
show_defeat(SHIP_NAME, cause)
```

```
process_destination(destination, oxygen, hull)
```

ocean Ai

These variables describe one ship.

They belong together, and they change together during play.

```
ship_name = "The Horizon"
crew = "A band of explorers"
oxygen = 100
hull = 100
```

// blueprint

Ship class

Names the attributes and methods every Ship has.

creates

// object

ship

One particular ship with its own current values.

CLASS A reusable description of one kind of thing.

STEP 01

Describe what every Ship stores.

Create player.py. Keep the class focused on the player ship.

player.py

```
class Ship:
    def __init__(self, name, crew, oxygen, hull):
        self.name = name
        self.crew = crew
        self.oxygen = oxygen
        self.hull = hull
```

ocean Ai

`class Ship:`

new type

Python learns what a Ship is.

`__init__`

setup method

Runs when a Ship is created.

`self`

this object

The particular ship being worked on.

STEP 02

Call the class to build a ship.

The four arguments fill the four setup parameters in order.

```
ship = Ship(
    "The Horizon",
    "A band of explorers",
    100,
    100,
)
```

ocean Ai

Use a dot to reach data on the object.

ship.oxygen means the oxygen belonging to this ship.

```
print(ship.name)
print(ship.oxygen)

ship.oxygen = ship.oxygen - 8
print(ship.oxygen)
```

ocean Ai

A method is a function that works on self.

The ship can check its own resources and display its own status.

`ship.is_defeated()`

reads oxygen and hull

`ship.show_status()`

prints current values

STEP 03

Move defeat and status into Ship.

Both methods read attributes through self.

```
class Ship:
    def __init__(self, name, crew, oxygen, hull):
        self.name = name
        self.crew = crew
        self.oxygen = oxygen
        self.hull = hull

    def is_defeated(self):
        if self.oxygen ⩽ 0:
            return "Oxygen depleted"
        if self.hull ⩽ 0:
            return "Hull destroyed"
        return None

    def show_status(self):
        print(f">> Oxygen: {self.oxygen} | Hull: {self.hull}")
```

ocean Ai

The object supplies self automatically.

Call with a dot. Do not pass the ship again.

```
ship = Ship("The Horizon", "A band of explorers",
92, 100)

ship.show_status()
cause = ship.is_defeated()
```

Expected: `>> Oxygen: 92 | Hull: 100`

// before

Loose values

oxygen, hull, ship_name

3+

// after

One object

ship

1

MEANING The code now matches the idea: these facts belong to one ship.

STEP 04

Import Ship, then create one object.

The class stays in player.py. In the real game, pass the four values from constants.

game.py

```
from player import Ship
```

```
ship = Ship(
    "The Voyager",
    "A curious crew from Seychelles",
    100,
    100,
)
```

// predict update the loop

Which loose variable disappears from this jump?

```
oxygen = oxygen - 8
```

→

```
ship.oxygen = ship.oxygen - 8
```

oxygen

The resource now lives on ship.

STEP 06

Process one object in place.

No tuple return is needed; the function updates ship attributes.

```
def process_destination(ship, destination):
    ship.oxygen, ship.hull, narration = process_encounter(
        destination, ship.oxygen, ship.hull
    )
    show_encounter(narration)

    if destination["has_water"]:
        ship.oxygen, ship.hull, water_narration =
process_water_planet(
            ship.oxygen, ship.hull
        )
        show_encounter(water_narration)
```

ocean Ai

State and behaviour travel together.

Update the ship, process the ship, ask the ship.

```
ship.oxygen = ship.oxygen - 8
process_destination(ship, asteroid)
ship.show_status()

cause = ship.is_defeated()
if cause:
    print(cause)
```

update → process → show → check

ocean Ai

Refactor to one Ship.

Use one class. Keep encounter classes for next week.

30 minutes

01 Create player.py with Ship.

02 Add four attributes and two methods.

03 Create ship in game.py.

04 Replace loose resource variables everywhere.

SUCCESS The game plays identically using one ship object.

// verify same game, better structure

Test the object through every important path.

UPDATE

One jump changes ship.oxygen.

DEFEAT

Oxygen and hull causes return correctly.

ENDING

Defeat and victory show ship.name.

Two class mistakes with clear clues

OBJECT MISSING

```
NameError: name 'missing_ship' is not defined
```

Check: Was the Ship object created first?

ATTRIBUTE MISSPELLED

```
AttributeError: 'Ship' object has no
attribute 'oxgyen'
```

Check: Match the attribute spelling in __init__.

Save the object pivot.

Both the new class and the refactored game belong in this checkpoint.

Terminal

```
$ git add player.py game.py
$ git commit -m "session 7: create Ship class"
$ git push
```

CHECK GitHub shows both changed files.

// reflect name the payoff

Why does ship fit the program better than separate values?

01 What does self mean?

02 What is the difference between a class and an object?

03 Why does is_defeated belong on Ship?

ocean Ai

// optional after the core

What else could your Ship remember?

Add a simple log list after the core—or move an intro display method onto Ship.

NEXT: encounters become interacting classes